Lessons Learned from the Census Bureau Grants Tool Capstone

Colin Wilkie, John Le, and Brandon Mistry

Introduction

Taking over a data application developed by another team is rarely straightforward. Our experience working on the Census Bureau Grants Tool — an app designed to help novice grant applicants find and visualize relevant Census statistics — was no exception. The journey from understanding legacy code to refining the app for real-world use presented several technical, organizational, and strategic challenges. This paper reflects on key learnings from our team's experience, focusing on onboarding with existing code, infrastructure adaptation, and laying the groundwork for a production-ready tool.

1. Challenges of Inheriting Legacy Code

One of the most immediate obstacles we faced was deciphering the prior team's codebase. While the original developers left behind a functional tool and some documentation, the project was built with numerous implicit assumptions:

- **File Path Hardcoding**: The app's scripts relied on hardcoded file paths, which made it difficult to run the code on machines other than the original developer's. Fixing this required building a config.json file to dynamically manage file paths across environments.
- **Inconsistent File Structure**: The data was spread across directories (e.g., "Data," "Discover," "Code"), with few comments explaining relationships. This made it hard to understand what files were necessary or how they interacted.
- **Missing Dependencies and Environment Drift**: Several packages listed in requirements.txt were outdated or unpinned, and dependencies for different scripts weren't well-synchronized. This caused repeated errors when trying to run the app outside Jupyter.

Despite these setbacks, the experience offered an important lesson: onboarding with legacy code requires a methodical approach. Our team created detailed documentation on how to replicate the environment, clarified missing dependencies, and standardized configuration to allow future users to deploy locally with minimal friction.

2. Technical Enhancements for Robustness

After achieving a basic working version, we focused on improving the app's stability and functionality. Our main contributions included:

- **Census API Integration**: We reviewed the Census API documentation and created secure access through .env files. We also developed helper functions to simplify data retrieval and began testing integration with new data sources such as the American Community Survey (ACS).
- **Dynamic Data Architecture**: Rather than relying on static CSVs (~16GB in total), we explored a more flexible pipeline where the tool can query and update datasets automatically. This included adding scripts to ingest and process data from the census-data-downloader GitHub package.
- **User-Centered Features**: Based on user personas (NGO writers, local officials, etc.), we sketched new features such as drill-down maps and grant-matching via NLP. While not yet implemented, these concepts guided backend decisions and UI/UX improvements.

This phase of the project underscored the value of modular design and version control. By encapsulating environment setup in a config file and using GitHub for version tracking, we laid the foundation for more maintainable code.

3. Strategic & Infrastructure Considerations

In discussions with our advisors and the Census Bureau, the question of where and how to host the app became a recurring theme. Several constraints shaped our thinking:

- **Local vs. Cloud Hosting**: Hosting locally provided full data access without API rate limits but made it harder to collaborate or scale. Cloud platforms like AWS or Ravenna were floated, though data size and cost remained concerns.
- **API Refresh Cycles**: Given that many Census surveys are annual, we determined that weekly or monthly refresh intervals were sufficient. However, any future production deployment would require a more formal data update strategy, potentially through scheduled ETL jobs.
- **Collaboration & Knowledge Transfer**: The previous team’s work was difficult to parse without direct communication. As such, we placed special emphasis on writing a robust “Getting Started” guide and README to ease onboarding for future contributors.

Ultimately, our infrastructure decisions favored local development for short-term velocity, while outlining a clear path to cloud deployment should the tool be institutionalized.

Conclusion

Our work on the Census Bureau Grants Tool provided a hands-on case study in what it takes to make legacy academic software production-ready. We encountered and addressed the common hurdles: undocumented code, environment inconsistencies, unclear file structure, and hosting ambiguity. In response, we prioritized reproducibility, code modularity, and documentation.

The outcome is not just a more functional app, but a repeatable framework for future teams to build upon. For anyone stepping into an inherited codebase, our biggest takeaway is this: don't just make it work, make it understandable, replicable, and ready for the next team.